

Deep-ELA: Deep Exploratory Landscape Analysis with Self-Supervised Pretrained Transformers for Single- and Multi-Objective Continuous Optimization Problems

Moritz Vinzent Seiler

moritz.seiler@uni-paderborn.de

Machine Learning and Optimisation, Paderborn University, Germany

Pascal Kerschke

pascal.kerschke@tu-dresden.de

Big Data Analytics in Transportation, TU Dresden, Germany;

ScaDS.AI Dresden/Leipzig, Germany

Heike Trautmann

heike.trautmann@uni-paderborn.de

Machine Learning and Optimisation, Paderborn University, Germany;

Data Management and Biometrics Group, University of Twente, Netherlands

Abstract

In many recent works, the potential of *Exploratory Landscape Analysis* (ELA) features to numerically characterize, in particular, single-objective continuous optimization problems has been demonstrated. These numerical features provide the input for all kinds of machine learning tasks on continuous optimization problems, ranging, i.a., from *High-level Property Prediction* to *Automated Algorithm Selection* and *Automated Algorithm Configuration*. Without ELA features, analyzing and understanding the characteristics of single-objective continuous optimization problems is — to the best of our knowledge — very limited.

Yet, despite their usefulness, as demonstrated in several past works, ELA features suffer from several drawbacks. These include, in particular, (1.) a strong correlation between multiple features, as well as (2.) its very limited applicability to multi-objective continuous optimization problems. As a remedy, recent works proposed deep learning-based approaches as alternatives to ELA. In these works, e.g., point-cloud transformers were used to characterize an optimization problem's fitness landscape. However, these approaches require a large amount of labeled training data.

Within this work, we propose a hybrid approach, *Deep-ELA*, which combines (the benefits of) deep learning and ELA features. Specifically, we pre-trained four transformers on millions of randomly generated optimization problems to learn deep representations of the landscapes of continuous single- and multi-objective optimization problems. Our proposed framework can either be used out-of-the-box for analyzing single- and multi-objective continuous optimization problems, or subsequently fine-tuned to various tasks focussing on algorithm behavior and problem understanding.

Keywords

Deep Learning, Exploratory Landscape Analysis, Single-Objective Optimization, Multi-Objective Optimization, Automated Algorithm Selection, High-level Property Prediction

1 Introduction and Related Work

Optimization problems, often found at the heart of numerous scientific and industrial applications, present challenges that necessitate robust and efficient problem-solving methodologies. A central concern in this field is the analysis and characterization of optimization problems' landscapes, i.e. continuous single objective problems but also i.a. combinatorial optimization, mixed-integer or multi-objective optimization. One methodological concept that has emerged as particularly significant for such a characterization is *Exploratory Landscape Analysis* (ELA; Mersmann et al., 2011). ELA facilitates a numerical representation of single-objective continuous optimization problems, providing insights into their intrinsic characteristics. The numerical features harvested from ELA serve as the foundation for various machine-learning tasks pertinent to continuous optimization. Such tasks include, but are not limited to, *High-level Property Prediction* (HL2P; Mersmann et al., 2011; Seiler et al., 2022; Volz et al., 2023), *Automated Algorithm Selection* (AAS; Rice, 1976; Kerschke et al., 2019), and *Automated Algorithm Configuration* (AAC; Hutter et al., 2009; Huang et al., 2019; Schede et al., 2022).

While ELA has undoubtedly transformed how researchers approach single-objective continuous optimization problems, it is not without its challenges. Two of the most pronounced limitations include the strong correlations between numerous features and their restricted use in the multi-objective domain. The advent of deep learning has ushered in a myriad of solutions across disciplines. Recent studies have proposed leveraging deep learning, specifically point-cloud transformers, as a potential remedy to the shortcomings of ELA (Seiler et al., 2022; Prager et al., 2022). However, such approaches, while promising, come with their own set of challenges, most notably the requirement of vast amounts of labeled training data.

Given the strengths and weaknesses of both, ELA and deep learning-based methodologies, there is a compelling case to be made for a synthesis of the two. This paper seeks to bridge this gap by introducing a hybrid approach, capitalizing on the merits of both paradigms. We thus present a novel method that employs four pre-trained transformer models tailored to extract deep representations of the landscapes of both single- and multi-objective continuous optimization problems. This innovative approach promises enhanced flexibility and efficacy of optimization problem analysis and understanding as these models can be used out-of-the-box without the need for additional feature selection or normalization. In addition, learned features do not require expertise to design meaningful feature sets. Instead, one can simply fine-tune Deep ELA on novel optimization problems that are not meaningfully represented by classical ELA.

We introduce these pre-trained transformers as *Deep Exploratory Landscape Analysis* (Deep-ELA).¹ The strengths of Deep-ELA are prominently seen in its inherent adaptability to multi-objective continuous optimization problems and the reduced correlation among its learned features. We demonstrate the straightforward adaptability of Deep-ELA by applying it to rigorous tests across three distinct case studies that showcase their efficacy:

1. A classification task involving problem instances from the *Black-box Optimization Benchmarking Suite* (BBOB; Hansen et al., 2009). Here, we focus on modeling and predicting their *High-Level Properties*, effectively revisiting the experiments previously conducted by Mersmann et al. (2011) and Seiler et al. (2022).

¹An extension of Deep-ELA to the pflacco package can be found here: https://github.com/mvseiler/deep_ela.git.

2. An assessment of single-objective AAS on BBOB, featuring twelve algorithms sourced from the *COMparing Continuous Optimisers* (COCO; Hansen et al., 2019) platform. This particular study echoes the works of Kerschke and Trautmann (2019) and later Prager et al. (2022).
3. An AAS study, applied to a set of multi-objective optimization problems, utilizing seven distinct algorithms. While this study draws inspiration from Rook et al. (2022), it ventures into a different direction, focusing on AAS rather than AAC.

The remainder of this paper is structured as follows. Initially, we equip readers with a concise background on ELA, providing an overview of feature-free alternatives and laying out the notation employed throughout the paper (Section 2). Subsequently, Section 3 dives into the architectural essence of Deep-ELA. Following this, we delve into a thorough exploration of the various datasets featured in this work in Section 4. We then pivot to a comparative analysis, contrasting the outcomes of Deep-ELA with traditional ELA across the previously mentioned case studies in Section 5. In Section 6, we present our conclusions and provide opportunities for future research.

2 Background

In the following, we will briefly outline the notation of this paper. An optimization function, Z , is given by:

$$Z : \mathcal{X} \rightarrow \mathcal{Y}, \quad \mathbf{x} \mapsto (z_1(\mathbf{x}), z_2(\mathbf{x}), \dots, z_m(\mathbf{x}))^\top \quad (1)$$

where $\mathcal{X} \subseteq \mathbb{R}^d$ is the decision and $\mathcal{Y} \subseteq \mathbb{R}^m$ the objective space. Every scalar function $z_i : \mathcal{X} \rightarrow \mathbb{R}$, with $i \in \{1, \dots, m\}$, symbolizes a distinct objective function that maps from the decision space $\mathcal{X}$ to a real value. In this context, $\mathbf{x} \in \mathcal{X}$ is referred to as a *candidate solution*, and the set $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \subseteq \mathcal{X}$ is formed of n such solutions. An objective or fitness vector is represented by $\mathbf{y} = Z(\mathbf{x})$ with $\mathbf{y} \in \mathcal{Y}$ and a length of m . If $m > 1$, then Z is a multi-objective function, whereas for $m = 1$, $\mathbf{y}$ is a scalar, and Z is a single-objective function. Finally, $Y = \{Z(\mathbf{x}) \mid \mathbf{x} \in X\}$ is a set of objective values.

In single-objective optimization, the task is defined as:

$$\mathbf{x}^* = \arg \min_{\mathbf{x} \in X} Z(\mathbf{x}) \quad (2)$$

Here, $\mathbf{x}^*$ is an optimal solution for Z . The goal of single-objective optimization is to find a solution $\mathbf{x}^*$ that w.l.o.g. minimizes the function Z globally. If multiple optimal solutions exist, Z is *multimodal*, and the aim shifts to locating all optimal solutions.

In contrast, in the multi-objective setting, optimizing for one objective can negatively affect another. Therefore, any solution provides a trade-off between the objectives, and the goal of multi-objective optimization is to find a set of optimal solutions, known as a non-dominated or Pareto set. Given two candidate solutions $\mathbf{x}_i$ and $\mathbf{x}_j$, Pareto dominance, denoted as $\mathbf{x}_i \prec \mathbf{x}_j$, means $\mathbf{x}_i$ dominates $\mathbf{x}_j$, if it is equal or better in all objectives, and strictly better in at least one of them. Formally, this is represented as:

$$z_k(\mathbf{x}_i) \leq z_k(\mathbf{x}_j) \quad \forall k \in \{1, \dots, m\}, \text{ and} \quad (3)$$

$$z_l(\mathbf{x}_i) < z_l(\mathbf{x}_j) \quad \exists l \in \{1, \dots, m\}. \quad (4)$$

The goal of multi-objective optimization is to determine the Pareto set

$$X_E = \{\mathbf{x}_i \in X \mid \nexists \mathbf{x}_j \in X : \mathbf{x}_j \prec \mathbf{x}_i\} \quad (5)$$

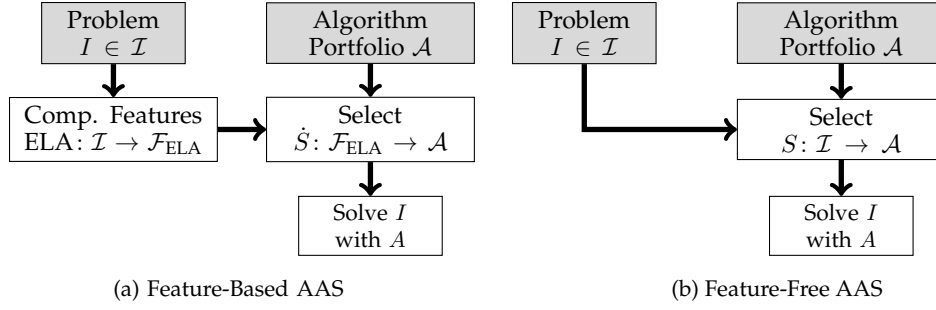


Figure 1: Comparison of the feature-based (left) versus feature-free (right) approach on one common downstream task: *Automated Algorithm Selection* (AAS). In the realm of AAS, there is no single universally superior algorithm for all problem instances. Instead, AAS uses a portfolio of algorithms $\mathcal{A} = \{A_1, \dots, A_{n_A}\}$. The optimal selector is formally defined as $S: \mathcal{I} \rightarrow \mathcal{A}$. Typically, the selector S is trained using machine learning to optimize a given performance metric. However, standard machine learners in AAS cannot process raw problem instances directly, necessitating a transformation into numerical vectors. This transformation is given as $F: \mathcal{I} \rightarrow \mathcal{F} \subseteq \mathbb{R}^{n_F}$, where F is a *mapper* converting an instance $I \in \mathcal{I}$ into a real-valued vector, termed (*instance*) *features*, in the *feature space* $\mathcal{F}$. Therefore, in a standard AAS scenario, the selector is defined as $\dot{S}: \mathcal{F} \rightarrow \mathcal{A}$, accepting features rather than actual instances.

and its mapping to the objective space, called Pareto front

$$Y_E = \{Z(\mathbf{x}) \mid \mathbf{x} \in X_E\}. \quad (6)$$

In many studies concerning *algorithm behavior* and *problem understanding*, Z is treated as a *blackbox* optimization problem, implying that there is no algebraic expression of Z given. Hence, no formal algebraic analysis can be conducted directly on Z . Instead, preliminary analyses are often carried out on a small sample of candidate solutions X , which is usually derived using a random or quasi-random generator like Latin Hypercube Sampling (LHS; McKay et al., 1979) or Sobol sequences (Sobol', 1967). The corresponding image Y is obtained by evaluating the optimization problem Z at the candidate solutions X . The tuple (X, Y) is then available for subsequent studies.

Such studies span *problem-* and *algorithm behavior-understanding*, *algorithm design*, and AAS. Hereafter, we will refer to these types of studies as *downstream tasks* or *downstream studies*. This terminology is borrowed from the *self-supervised* training community, which differentiates between two phases: (1.) *pre-training* and (2.) *fine-tuning*. Self-supervised learning addresses tasks with insufficient training datasets. Models are thus pre-trained using an auxiliary training task – potentially using a synthetic set if the primary training set is inappropriate (e.g., too small or risk of overfitting). Subsequently, these models are applied or fine-tuned for the original *downstream task*.

2.1 Exploratory Landscape Analysis

For downstream tasks involving continuous optimization problems, traditional machine learning methods aim to learn a specific underlying task. In AAS, for instance, classical machine learners like *Support Vector Machines* (SVM; Cortes and Vapnik, 1995), *Random Forests* (RF; Breiman, 2001), or *k-Nearest Neighbors* (*k*NN; Cover and Hart, 1967),

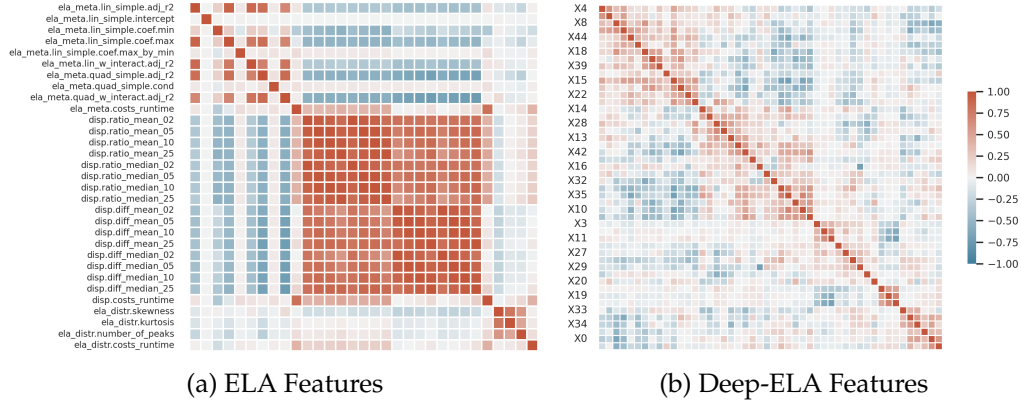


Figure 2: Comparison of exemplary correlation matrices of (Deep-)ELA features on BBOB: (a) classical ELA features (here: *meta-model*, *dispersion* and *y-distribution*), and (b) Deep-ELA features (Large-50d). Correlations are calculated across all 24 functions of the BBOB suite, but individually for every instance 1 to 20 and dimension 2, 3, 5, 10. Afterward, the 80 correlation maps are mean-aggregated.

are utilized to address the *Algorithm Selection Problem* (ASP) (see Figure 1a). The difference between AAS and ASP is that while the latter refers to the challenge of choosing the best algorithm from a set of algorithms for a specific instance, based on performance metrics like speed or solution quality; the former addresses the use of machine learning to automatically select the best algorithm, typically based on problem features. However, these classical learners cannot directly process raw samples of candidate solutions, i.e., (X, Y) , as the learners necessitate feature vectors as input per problem instance. Yet, transforming the set of (randomly distributed) points into a vector is not suitable, as such a mapping either (a) lacks an unambiguous order of the points, or (b) requires the points to be aligned in a grid structure, which in turn is affected by the *curse of dimensionality*. Therefore, it is essential to convert the set of candidate solutions (X, Y) into a vector of numerical values that provides a meaningful representation of the candidate tuple. This conversion, or transformation, is performed by a mapping function, or short *mapper*, $F : (\mathcal{X}, \mathcal{Y}) \rightarrow \mathcal{F} \subseteq \mathbb{R}^{n_{\mathcal{F}}}$, which translates the decision and objective values into a fixed-length real-valued vector, termed (*instance*) *features*, in the *feature space* $\mathcal{F}$.

For continuous optimization problems, *Exploratory Landscape Analysis* (ELA; Mersmann et al., 2011) features are commonly employed in many downstream tasks, including AAS. ELA was termed by Mersmann et al. (2011) but some feature sets that nowadays are considered as subsets of ELA were already introduced before, i.e. *Dispersion* (Lunacek and Whitley, 2006) and *Fitness Distance Correlation* (Jones et al., 1995). Formally, ELA is a function that maps samples (X, Y) from the decision and objective space $(\mathcal{X}, \mathcal{Y})$ to the c_{ELA} -dimensional feature space $\mathcal{F}_{\text{ELA}} \subseteq \mathbb{R}^{c_{\text{ELA}}}$

$$\text{ELA} : (\mathcal{X}, \mathcal{Y}) \rightarrow \mathcal{F}_{\text{ELA}}.$$

Over time, a plethora of ELA features for single-objective optimization problems emerged (Kerschke and Preuss, 2023), such as:

1. **Classical ELA** (Mersmann et al., 2011) is a superset of six different groups of ELA features, of which the *meta-model*- and *y-distribution*-features are the most com-

monly used ones.

2. **Dispersion** (Lunacek and Whitley, 2006) measures the spread in decision space of the best fraction of points in relation to the spread of all points of the full sample.
3. **Information Content** (Muñoz et al., 2014) measures, i.a., the landscape’s smoothness and ruggedness based on statistics summarizing a series of random walks.
4. **Fitness Distance Correlation** (Jones et al., 1995) measures the correlation between distances in the objective and distances in the decision space.
5. **Nearest Better Clustering** (Kerschke et al., 2015) compares the relation between the set of nearest neighbors in the decision space to the set of nearest better neighbors, where ‘better’ relates to neighbors with lower objective values.
6. **Miscellaneous** (Kerschke and Trautmann, 2019) is a collection of different feature types such as features based on *Principal Component Analysis* (PCA).

Though ELA features are frequently used, they also have some limitations (Kerschke and Preuss, 2023). As, e.g., discussed by Renau et al. (2019) and Eftimov et al. (2020), many ELA features exhibit strong correlations to each other, as illustrated in Figure 2a, or have a low *Signal to Noise Ratio* (SNR), see Figure 3. Moreover, some features from sets like *Classical ELA*, *Information Content*, and *PCA* are sensitive to random scaling and shifting (Renau et al., 2020); see Prager and Trautmann (2023) for an in-depth analysis of ELA feature sensitivity. These issues necessitate *feature selection* for many downstream tasks to eliminate noise or redundancy. Finally, ELA features are primarily tailored for single-objective optimization problems, not multi-objective ones. In fact, Kerschke and Trautmann (2016) used ELA features to characterize bi-objective problems; however, they only considered simple test problems from DTLZ (Deb et al., 2005) and ZDT (Zitzler et al., 2000), and applied ELA to the single-objective components of these problems, neglecting any interaction between the objectives.

Another use-case for ELA features is *Instance Space Analysis* (ISA; Smith-Miles et al., 2014). It can be used to analyze the space that is covered by all instances within a given dataset to identify (dis-)similarities of structural properties between instances. Smith-Miles and Muñoz (2023) propose a framework that utilizes ELA features for ISA. On the other hand, ISA can also be used to identify empty regions that are not covered by existing optimization problems. (Prager et al., 2023) proposed a method to create artificial benchmark problems that cover specific regions within the instance space. These problem instances are learned by neural networks.

2.2 Deep Learning-Based Approaches as Alternative to ELA

In the *evolutionary computation* (EC) community, methods that avert features are often termed *feature-free*. Yet, in the deep learning domain, this term can be misleading. In the latter, *features* concern internally learned features of complex deep learning models, whereas in the EC domain, (ELA) features refer to summary statistics quantifying the intrinsic characteristics of the given optimization problem (see Figure 1b for an illustration of feature-free AAS).

In recent studies, efforts have been made to solve downstream tasks without depending on ELA features, thus, sidestepping their limitations. In the context of discrete optimization (which is beyond the scope of this paper), Alissa et al. (2019) employed a *Long Short-Term Memory* (LSTM; Hochreiter and Schmidhuber, 1997) for AAS on the 1d-Bin Packing problem, without pre-computed features. Similarly, Seiler et al. (2020) used *Convolutional Neural Networks* (CNN; LeCun and Bengio, 1995) for AAS on the *Traveling Salesperson Problems* (TSP) without pre-computed instance features.

Within continuous optimization, Prager et al. (2021) introduced a method applying

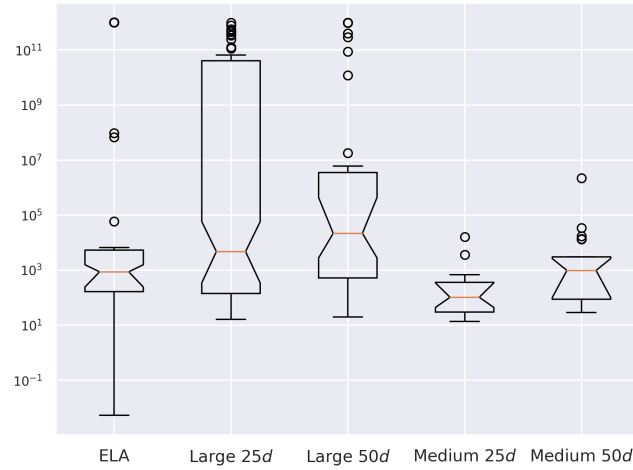


Figure 3: *Signal to Noise Ratio (SNR) of ELA features (left boxplot) on BBOB compared to the SNR values of features from four Deep-ELA models. We used $\text{SNR} = \mu^2/\sigma^2$ with mean μ and standard deviation σ . For $\sigma \simeq 0$, values are imputed with 10^{12} , which is the highest observed value. Higher values indicate lower noise. SNR values are calculated per feature based on instances 1 to 20 and then mean-aggregated over the 24 functions and four dimensions (2, 3, 5, 10). Notches show the 95% confidence intervals around the median. The large models yield the highest SNR while the medium models yield the lowest which is to be expected as the large models contain more parameters to create more sophisticated features. Classical ELA features are somewhat ‘in-between’ while simultaneously containing features with multiple, very low SNR values.*

CNNs to two-dimensional single-objective optimization problems. This was expanded by Seiler et al. (2022), who presented image- and point cloud-based techniques for high-dimensional, single-objective optimization problems. Notably, the *Point Cloud Transformer* (PCT; originally proposed by Guo et al., 2021) delivered top-tier results without the need for ELA features. Subsequently, Prager et al. (2022) employed PCT alongside traditional ELA-based methods for AAS on continuous, single-objective optimization tasks. Only ELA-based *Multi-Layer Perceptrons* (MLP) with a carefully designed loss function managed to surpass the performance of the PCTs.

Compared to deep learning-based feature-free techniques, ELA-based methods usually require significantly fewer training instances under the assumption that these smaller benchmark sets are nonetheless representative and comprehensive. This becomes particularly advantageous when optimization problems are scarce or crafting a training dataset is computationally expensive. While data augmentation can amplify the training set, deep learning methods may still generalize poorly in case of limited training data. Note that in this context, *small* means a few hundred and *large* implies tens or even hundreds of thousands of instances; e.g. Seiler et al. (2022) utilized 144 000 training examples while Prager et al. (2022) only relied on 4 800 training examples (both including ten repetitions per instance).

3 Deep Exploratory Landscape Analysis

In the following, we aim to merge the advantages of both *feature-based* and *feature-free* analysis in the context of continuous optimization. We introduce a large, pre-trained deep learning model trained on millions of randomly generated continuous, single- and multi-objective optimization instances. The generator for these tasks draws inspiration from Tian et al. (2020) and van Stein et al. (2023), but with adaptations to suit our requirements. Our model’s general goal is the automated extraction of instance features from initial samples of the given optimization instance. We dub this methodology *Deep Exploratory Landscape Analysis* (Deep-ELA). Seiler et al. (2024) analyzed these learned features in detail and, especially, analyzed the complementarity and synergies between classical and Deep ELA.

3.1 Outline of Deep ELA

Deep-ELA is trained to be invariant against scaling, shifts, and rotations, yielding higher SNR values and minimally correlated features (see Figure 2b and Figure 3) with high importance. Hence, feature selection becomes less important — though still potentially valuable in some cases, see Seiler et al. (2024) — as will be demonstrated in the experiments (see Section 5). To achieve this, a self-supervised learning task is employed, which educates the model to formulate a representative and unique feature vector for an optimization instance – a process often termed *feature-learning*. Importantly, Deep-ELA is not restricted to single-objective optimization problems and can naturally be applied to multi-objective optimization problems. We will showcase Deep-ELA’s competitive edge against the traditional ELA but also feature-free approaches in existing studies (e.g., Seiler et al., 2022; Prager et al., 2022). We also applied our model to a bi-objective case analogous to Rook et al. (2022). See Section 5 for an outline of all the case studies.

To the best of our knowledge, only two alternative approaches, termed DoE2Vec (van Stein et al., 2023) and TransOpt (Cenikj et al., 2023), parallels our proposed method. However, DoE2Vec varies substantially from Deep-ELA: its model, rooted in a basic autoencoder design, solely focuses on the objective space $\mathcal{Y}$, completely disregarding the decision space $\mathcal{X}$. This reduced informational scope results in a markedly inferior performance compared to the approaches by Seiler et al. (2022). Additionally, the proposed method is not invariant to the order of objective values in $\mathcal{Y}$, potentially affecting its efficacy. Further, Cenikj et al. (2023) propose also Transformer models, TransOpt, that were trained on an AAS task but the learned (hidden) features can also be used as supplementary for classical ELA features. Contrary to Deep-ELA, the learned features are specific to the underlying AAS task and may lack generalization to other downstream tasks.

In terms of Deep-ELA’s architecture (refer to Figure 4), we predominantly adhered to the traditional transformer encoder blueprint as suggested by Vaswani et al. (2017). Contrary to the PCT design by Guo et al. (2021) and Seiler et al. (2022), we have embedded extra feed-forward modules to align more closely with the original transformer. According to a recent study (Geva et al., 2020), the feed-forward layers in transformers act like a memory that stores learned patterns; lower layers store more basic patterns while higher layers store more complex ones. In our case, we expect Deep-ELA to learn and memorize certain patterns of optimization problems. Subsequently, we adopted pre-normalization as recommended by Nguyen and Salazar (2019) and used *Gated Linear Units* (GLU; Dauphin et al., 2017) as the activation function between all feed-forward modules to reduce the training time as demonstrated by the authors. For

the *feature extractor* (the last linear layer), we opted for the Tanh activation, mapping all activations to $[-1, 1]$ and, hence, dispensing the need for feature normalization in later tasks.

The final model, *Deep-ELA*, can be formally expressed as

$$\text{Deep-ELA: } (\mathcal{X}, \mathcal{Y}) \rightarrow \mathcal{F}_{\text{D-ELA}} \quad (7)$$

with $c_{\text{D-ELA}}$ -dimensional feature space $\mathcal{F}_{\text{D-ELA}} \subseteq \mathbb{R}^{c_{\text{D-ELA}}}$. Analogous to ELA, Deep-ELA requires a sample of $(X, Y) \in (\mathcal{X}, \mathcal{Y})$ as input. The resulting features lie within $[-1, 1]$ and exhibit minimal correlation and redundancy. This contrasts with features from $\mathcal{F}_{\text{ELA}}$, which are un-normalized and often redundant (see Figure 2 for a comparison).

During the training phase, X is drawn randomly uniformly within the bounded decision space $\mathcal{X}$. We favored uniform sampling over quasi-random sampling strategies to (1.) reduce additional computational overhead during the training procedure, (2.) reduce Deep-ELA's dependence on a particular sampling method, and (3.) enhance the challenge of the self-supervised training task (which is described in more detail in Section 3.2.2). After pre-training, more advanced sampling methods, such as LHS and Sobol sampling, can be employed to improve coverage of the decision space.

Deep-ELA can accommodate any sample length of (X, Y) but has a constraint regarding the sum of the decision and objective space dimensions: $d + m \leq \nu$. Here, ν is an additional hyperparameter of Deep-ELA, termed the *degree of dimensionality*. It specifies the maximum combined dimensionality of $\mathcal{X}$ (denoted by d) and $\mathcal{Y}$ (denoted by m). Deep-ELA is versatile and can handle varying dimensionalities $d \in [1, \nu]$ and $m \in [1, \nu]$, provided their combined dimensionality does not surpass ν . This flexibility extends to multi-objective optimization problems since the model can accept $m \geq 1$. It is worth noting that the model was trained on problem instances with $d \geq 2$ and $m \geq 1$.

3.2 Model Structure

Our proposed Deep-ELA model is depicted in Figure 4. The workflow begins with the input $(X, Y) \in (\mathcal{X}, \mathcal{Y})$, which undergoes a *k-Nearest-Neighborhood* ($k\text{NN}$) embedding as illustrated in Figure 6. This embedding methodology, originally introduced by Seiler et al. (2022), seeks to incorporate the local neighborhood of every $\mathbf{x} \in X$. Given that transformers excel at capturing global patterns, but may overlook local nuances, the $k\text{NN}$ embedding ensures local information is not ignored. Post-embedding, the encoding undergoes a projection to the deep learning network's hidden number of features, typically denoted as d_{model} (refer to Section 3.2.1). Subsequently, every member of $\mathbf{x} \in X$ alongside its k nearest neighbors is termed as *tokens*, aligning with the terminology conventionally associated with transformers.

Following this linear projection, the sequence of tokens is sequentially processed by six combined modules of *Multi-Head Attention* (MHA) and *Feed-Forward* (FF), concluding with a final *Layer Normalization* (LN; Ba et al., 2016). Through experimentation, we determined that six iterations strike an optimal balance between model intricacy and performance. This choice was also supported by Vaswani et al. (2017) as they also opted for six iterations. The output post-LN layer manifests as a normalized $(n \times d_{\text{model}})$ -dimensional tensor, with n representing the number of tokens. To derive the features $\mathcal{F}_{\text{D-ELA}}$, the final embedding is condensed via a linear layer with GLU activation, referred to as *feature-extractor*, into a $\mathbb{R}^{n \times c_{\text{D-ELA}}}$ tensor. Ultimately, a mean pooling step across the token sequence yields $c_{\text{D-ELA}}$ features, which are constrained within $[-1, 1]$ by a concluding Tanh activation. An illustration of the model's topology and an overview of its associated hyperparameters are given in Figure 4 and Table 1, respec-

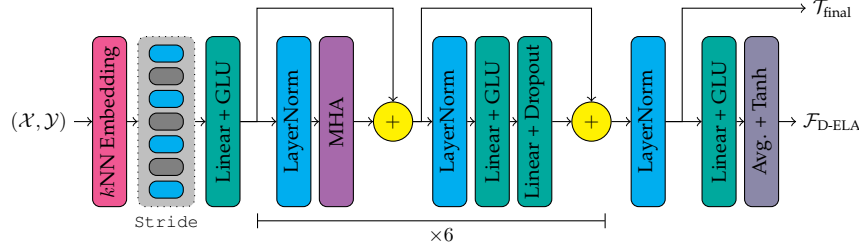


Figure 4: Illustration of the chosen topology of the backbone model without the training heads. The model receives $(\mathcal{X}, \mathcal{Y})$ as input and outputs $\mathcal{F}_{\text{D-ELA}}$ – and, optionally, the embedding of the tokens $\mathcal{T}_{\text{Final}}$ after the final LayerNorm. $\mathcal{T}_{\text{Final}}$ is only relevant for the contrastive loss during training and is ignored after training. The initial k NN embedding (Seiler et al., 2022) is used to capture the local information of all points from the input sample, and followed by a stride operator to optionally reduce the number of tokens without losing information. Next, the model consists of six Multi-Head Attention blocks, followed by a Feed-Forward block of two successive Linear layers each. The LayerNorm layers are after the shortcuts as proposed by Nguyen and Salazar (2019). We chose GLU activations in the Feed Forward layers with a $4\times$ larger number of hidden neurons. The last Linear + GLU layer projects the high-dimensional embeddings into lower dimensions. Afterward, the mean over all tokens is computed and normalized into $[-1, 1]$ by a Tanh activation.

tively.

This architecture termed the *backbone model*, is versatile for diverse downstream tasks. However, as the backbone essentially functions as a feature generator, it lacks trainability. Consequently, to facilitate training, we incorporate (1.) two *feature-heads* (distinguishing one as a so-called *student* and the other one as a *teacher*), and (2.) an additional teacher feature-extractor, as visualized in Figure 5. The student components are updated via *gradient descent*, while the teacher counterparts are updated via *exponential moving average* (EMA). Both are integral for the self-supervised loss computation. The training loss strategy revolves around providing distinct, augmented versions of the same objective instance to both the teacher and student. Here, the teacher generates target projections from which the student gleans insights. However, we refrained from using two backbone models (one for the student and one for the teacher) as this would drastically increase the total amount of model parameters. So, the backbone model is the same for both the student and teacher. More details about the loss function and the training routine are provided in Section 3.2.2.

Moreover, our approach employs the *InfoNCE* (Oord et al., 2018) loss function, specifically tailored for self-supervised learning. Notably, our model is strongly influenced by the principles of *Momentum Contrast for Unsupervised Visual Representation Learning* (MoCo; Chen et al., 2021) in its third iteration. Diverging from MoCo V3, we abstained from using a concluding MLP predictor, having observed its detrimental impact on performance and convergence in our context. Instead, our choice was a concluding *Batch Normalization* (BN; Ioffe and Szegedy, 2015) with running mean and standard deviation but without the scale and shift parameters as we aim for z -standardization of the output projection. Even though the Batch Normalization layer did not notably elevate our performance, it amplified differences between dissimilar functions within the feature space, making distinctions more pronounced.

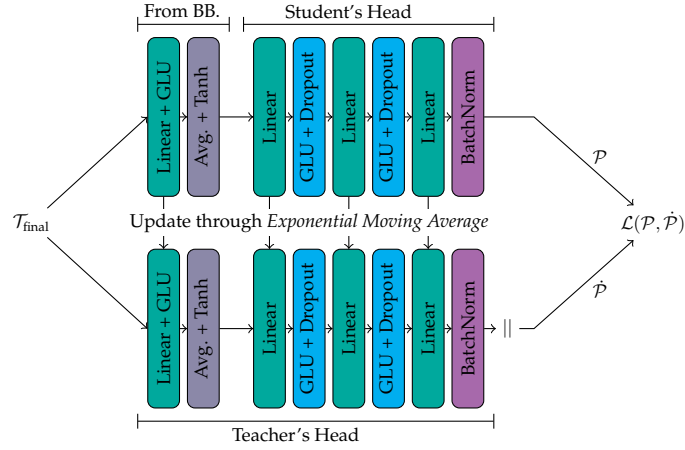


Figure 5: The two training heads on top of the backbone model. Note that the first two layers before the student’s head are part of the backbone model (*from BB.*). Both heads are removed after training. The student’s head gets updated by gradient descent while the momentum head is an old version of the student’s head and gets updated through EMA. The design follows closely the idea of Chen et al. (2021).

Given their pivotal roles in our model, the k NN embedding layer and the contrastive loss function will be described in more detail in the subsequent sections.

3.2.1 k NN Embedding

Our Deep-ELA model, schematically represented in Figure 4, operates in two steps: input processing and embedding. Both are outlined in the following.

Input Processing: The input sets (X, Y) undergo individual z -standardization, both per set and per dimension. To extend their dimensionality, they are padded with zeros resulting in $X', Y' \in \mathbb{R}^{n \times \nu}$. They are then combined into a single set, $\hat{T} = X' \parallel Y'$, where $\parallel$ indicates the element-wise concatenation operator. Following this procedure, $\hat{T} \subseteq \mathbb{R}^{n \times 2\nu}$ is structured such that its first ν dimensions are always decision values, while the subsequent ν dimensions represent objective values. Although this approach results in a matrix where at least half of the entries are zeros (given the dimensionality constraint $d + m \leq \nu$), it offers two advantages compared to using an additional indicator vector that discriminates between decision and objective values: (1.) all input variables remain real-valued, eliminating a mix of numeric and categorical values (that an identifier would introduce); (2.) adding an indicator vector of length ν would also produce an input of dimensionality $n \times 2\nu$.

Embedding: For each $(\mathbf{x}, \mathbf{y}) \in (X, Y)$, its $k - 1$ nearest neighbors are identified, projected to its local neighborhood, and then concatenated to $(\mathbf{x}, \mathbf{y})$, resulting in:

$$\mathbf{t} = (\mathbf{x}, \mathbf{y}, (\mathbf{x}_1 - \mathbf{x}), (\mathbf{y}_1 - \mathbf{y}), \dots, (\mathbf{x}_{k-1} - \mathbf{x}), (\mathbf{y}_{k-1} - \mathbf{y}))^T. \quad (8)$$

The tokens $\mathbf{t} \in T$ form a vector with $T \subseteq \mathbb{R}^{n \times 2k\nu}$. Every element in (X, Y) is termed as the *global context* due to its absolute position in the decision and objective spaces. Conversely, the $(k - 1)$ nearest neighbors represent the *local context*, defined by their relative positions to each $(\mathbf{x}, \mathbf{y})$. This distinction is pictorially elaborated in Figure

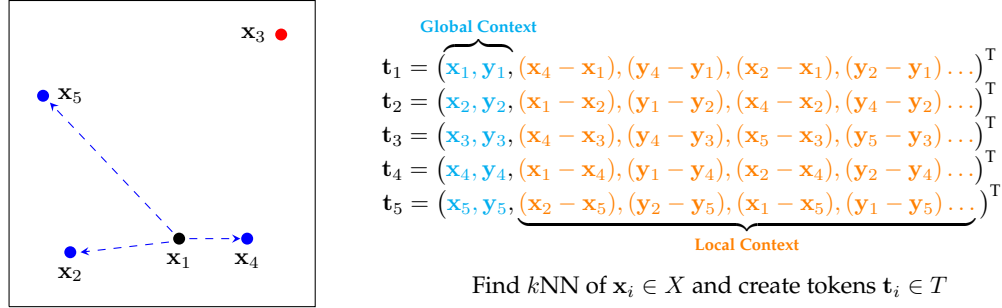


Figure 6: For every $\mathbf{x}_i \in X$ in the decision space, the $(k - 1)$ nearest neighbors are identified and together with their respective objective values combined into a single vector. Next, the $(k - 1)$ nearest neighbors are projected into the local neighborhood but centered by $\mathbf{x}_i$ and $\mathbf{y}_i$, i.e., subtracting them from all its $(k - 1)$ nearest neighbors. We refer to $(\mathbf{x}_i, \mathbf{y}_i)$ as the global context and to all its $k - 1$ neighbors as the local context.

6. The embedding process can be formally outlined as:

$$k\text{NN-Emb.}: (X, Y) \rightarrow \mathcal{T} \subseteq \mathbb{R}^{n \times 2k\nu}. \quad (9)$$

Next, $\mathcal{T}$ undergoes a linear projection, followed by a token-wise applied GLU activation, mapping the points to a higher-dimensional space $\mathbb{R}^{n \times d_{\text{model}}}$ (see Figure 4).

3.2.2 Contrastive Loss

As previously highlighted, the model employs two distinct heads during training. The *student* head updates via gradient descent, while the *teacher* head employs *exponential moving average* (EMA) for updates. The outputs of these heads are termed as the *online-projection* $\mathcal{P} \subseteq \mathbb{R}^{8 \cdot \text{CD-ELA}}$ (from the student), and the *target-projection* $\dot{\mathcal{P}} \subseteq \mathbb{R}^{8 \cdot \text{CD-ELA}}$ (from the teacher). The eight-fold dimensionality of the projections increases the heads' flexibility in aligning their representations. This methodology takes inspiration from Grill et al. (2020), who also considered higher dimensionalities for their heads.

The model's training closely mirrors the principles of MoCo V3, including the adoption of the InfoNCE loss function, as introduced by Oord et al. (2018). It has shown state-of-the-art results in other feature-learning endeavors, as evidenced by Grill et al. (2020) and Chen et al. (2021). Given a batch of online-projections, $P = \{\mathbf{p}_i \in \mathcal{P} \mid \forall i \in \{1, \dots, j\}\}$, and a batch of target-projections, $\dot{P} = \{\dot{\mathbf{p}}_i \in \dot{\mathcal{P}} \mid \forall i \in \{1, \dots, j\}\}$, with j being the batch-size, the overall goal of the InfoNCE is to fulfill the following equation:

$$\sigma(P\dot{P}^\top) \stackrel{!}{=} I.$$

Here, σ is the *Softmax* activation, and I is the identity matrix. Broadly speaking, the loss function aims to maximize the covariance between an instance's online- and target-projection and to minimize it between different instances. The InfoNCE loss function is formulated based on the *cross-entropy*:

$$\mathcal{L}_{\text{InfoNCE}}(\mathcal{P}, \dot{\mathcal{P}}; \tau) = 2\tau \cdot H\left(\sigma\left(\frac{\mathcal{P} \times \dot{\mathcal{P}}^\top}{\tau}\right)\right) + D_{\text{KL}}\left(\sigma\left(\frac{\mathcal{P} \times \dot{\mathcal{P}}^\top}{\tau}\right), I\right). \quad (10)$$

In this equation, H and D_{KL} denote the *entropy* and the *Kullback-Leibler divergence*, respectively. The cross-entropy approach provides pronounced gradients, even for vanishing activations, compared to the e.g. *mean squared error* (MSE). The loss function also

depends on the critical hyperparameter τ , which is the temperature of the `Softmax` activation σ . Usually, the value of τ is set in the range $(0, 0.3]$. The smaller τ is, the higher the loss for *hard negative samples*, i.e., distinct instances that yield similar projections and, thus, are *hard* for the model to differentiate. On the contrary, for larger values of τ , the loss is more evenly spread over hard and easy negative samples. Through initial testing, we found that $\tau = 0.05$ works well in our scenario. As an explanation, we argue that small τ -values work better in our scenario as we rely solely on the stochasticity of the synthetic data generator. We cannot control the characteristics of the generated instances. So it is very likely that most instances are easy to distinguish and hard negatives occur rather infrequently. To account for this, we opted for two solutions: (1) a small τ -value will penalize hard negatives effectively also in large batches and (2) utilizing large batches to increase the likelihood of the appearance of hard negative samples within a single batch.

During training, the model receives two augmented pairs of $(X_1, Y_1), (X_2, Y_2)$ for every problem instance Z in a batch of j problem instances. For every batch, the student and also the teacher process both augmentations of every instance, yielding four distinct projections: $P_1, P_2 \in \mathcal{P}$ (from the student) and $\dot{P}_1, \dot{P}_2 \in \dot{\mathcal{P}}$ (from the teacher). The overall loss is computed as:

$$\mathcal{L} = \frac{1}{2} (\mathcal{L}_{\text{InfoNCE}}(P_1, \dot{P}_2; \tau) + \mathcal{L}_{\text{InfoNCE}}(\dot{P}_1, P_2; \tau)). \quad (11)$$

The deliberate pairing of $(P_1, \dot{P}_2)$ and $(\dot{P}_1, P_2)$ aims to maximize the loss across both augmented versions. For augmentation, we adopted rotations and inversions of decision variables and randomized the sequence of decision and objective variables. As the augmentations do not alter the underlying optimization problem, the predicted features should be invariant to these changes. Furthermore, the z -standardization in the embedding layer ensures the model remains invariant to scale and shift modifications.

4 Datasets

In total, we considered four sets of optimization problem suites for training: single-objective problems from the *Black-Box Optimization Benchmarking Suite* (BBOB; Hansen et al., 2009), multi-objective instances from the *Biobjective-BBOB Suite* (Bi-BBOB; Brockhoff et al., 2022), selected instances from the R-package `smooft2` (*Single- and Multi-Objective Optimization Test Functions*), and an adapted version of the random function generator as introduced by Tian et al. (2020).

4.1 Black-Box Optimization Problems

Most validation and testing optimization problems come from BBOB, provided by the *COmparing COntinuous Optimisers* (COCO; Hansen et al., 2019) platform. COCO offers a diverse range of continuous optimization problem suites, spanning from single- and multi-objective to noisy and mixed-integer problems. This study primarily engaged with the 24 noiseless, single-objective functions from the (classical) BBOB suite. For each of these functions, 1 000 unique instances were produced, of which the first 500 served for testing and the latter for validation purposes (see Section 5). Function dimensionalities were chosen from $d \in \{2, 3, 5, 10\}$. Although the suite includes functions with $d \in \{20, 40\}$, our model’s current dimensional constraints prevent their inclusion.

²<https://github.com/jakobbossek/smooft>

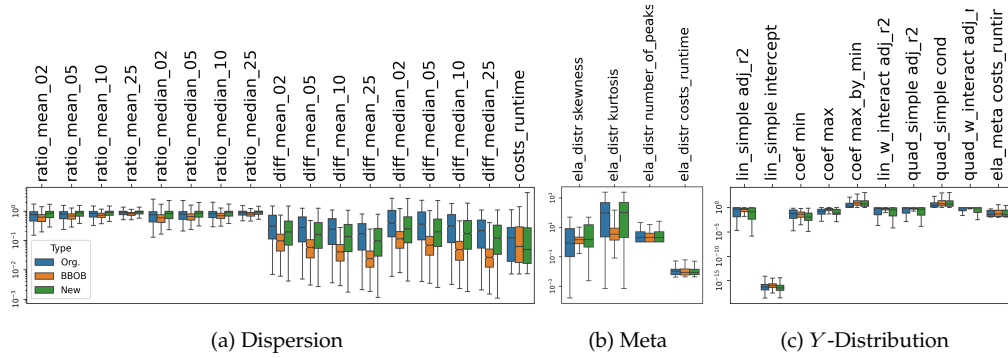


Figure 7: Distributions of feature values for three exemplary sets of ELA features, differentiated by three types of test problems – the original instance generator by Tian et al. (2020) (blue), the BBOB suite (orange), and our adapted generator (green). For all problems, we used dimensions 2, 3, 5, 10. In the case of BBOB, we considered all 24 noiseless functions with instances 1 to 20. For the random generators, we used the same number of instances (in total).

For the multi-objective case study, we analyzed the same multi-objective problems that Rook et al. (2022) examined. These bi-objective ($m = 2$) functions have a decision space dimensionality of $d = 2$. Specifically, we considered the Bi-BBOB functions f_{46} , f_{47} , and f_{50} , along with the ZDT, DTLZ, and MMF function groups (excluding ZDT5 and MMF13) from the `smooF` package. Further details are given in Section 5.4.

4.2 Random Optimization Problems

Tian et al. (2020) introduced a method capable of generating an arbitrary number of optimization problems in milliseconds. This approach hinges on a random tree-like structure constituted of various operators such as `mean`, `sum`, `exp`, `log`, among many others. This generator operates on three input parameters: the number of dimensions of the decision space (d), and the lower and upper bounds of the number of operators. Once initiated, the generator fabricates an optimization problem for a d -dimensional set of decision variables $\mathcal{X}$ by randomly assorting a number of operators bounded by the provided lower and upper bounds. The result is a randomly constructed optimization problem that yields the objective values Y . We manually calibrated the upper and lower bounds of the generator to craft optimization problems that are similar to BBOB. We found that when choosing these bounds too small or too large, the generator produces objectives that are too simple or too complicated in comparison to BBOB. Yet, as the models (like any other machine learner) can only generalize to instances that are similar to the training data, we need to ensure that BBOB is not out-of-distribution to the models. We found that upper and lower bounds of (4, 32) fulfill this criterion well. Additionally, we incorporated extra operators to amplify the linear and squared dependencies among decision and objective variables, noting that the original generator somewhat diverged from the BBOB suite in these aspects. Furthermore, we tweaked the skewness and kurtosis of the objective variables for closer alignment with BBOB (refer to Figure 7). Last, the original `Python` implementation yielded NaN-values quite often. We identified and modified those operators that cause the appearance of NaN-values. Afterward, we observed that the generator produces diverse optimization

problems that are more similar to BBOB in terms of their complexity of ELA features (see Figure 7). Yet, it shall be noted that the random functions are not identical to BBOB. Instead, the random functions cover similar characteristics as BBOB.

However, given the inherent randomness of the generator, not all produced optimization problems are applicable for training. We thus dismiss any instance that does not satisfy the following three conditions:

1. The output Y must be a set of scalars.
2. The objective values must have a standard deviation of at least 0.1
3. The objective values must lie within $[-10\,000\,000, +10\,000\,000]$.

To generate multi-objective instances, we randomly designed m single-objective functions and combined them into a multi-objective instance. While this methodology proved satisfactory for our purposes, the combined set of random single-objectives may not fully encapsulate the inherent traits of multi-objective problems. Notably, such problems often harbor interacting objectives, potentially exhibiting mutual dependencies. However, randomly creating multi-objective instances with such interdependencies proved to be much more challenging. We thus postponed this endeavor for future work, and hope that randomness will generate objectives with mutual dependencies. We built upon the Python implementation³ by van Stein et al. (2023).

5 Experiments

In the following sections, we describe and analyze the pre-training of our Deep-ELA models. Subsequently, we examine the results of three case studies. The first two, *High-Level Property Prediction* (Seiler et al., 2022) and *Single-Objective Automated Algorithm Selection* (Prager et al., 2022), have been conducted in previous studies. Please note that we directly took the results from previous work of Seiler et al. (2022); Prager et al. (2022); van Stein et al. (2023). The third case study, i.e., the *Multi-Objective Automated Algorithm Selection*, is adapted from Rook et al. (2022) and introduces a newly created dataset.

5.1 Pretraining

Before training our final models, we conducted several preliminary studies to determine the optimal hyperparameters and model topologies for our use cases. We experimented with various τ values, momentum factors, numbers of heads, and layers. Eventually, we systematically selected the hyperparameters presented in Table 1.

Given the availability of two high-performance computing (HPC) servers – one with three NVidia Quadro RTX 6000 GPUs and the other with three NVidia RTX A 6000 GPUs – we decided to train the larger models on the RTX A 6000 GPUs, which have 48GB of GPU memory, compared to the Quadro’s 24GB. The smaller models were comfortably trained on the 24GB GPUs. We maximized the batch size, retaining a small buffer, to include as many samples as possible within each batch as the used loss function benefits from large batches. Each model was trained across three GPUs, handling three batches concurrently. After the forward pass, the three batches were merged into one joint batch for loss computation, effectively tripling the batch size and increasing the likelihood of the appearance of hard negative samples.

We chose to train both a medium- and a large-sized model. The medium model can manage a total dimensionality of six, while the large model can accommodate up

³<https://www.github.com/Basvanstein/doe2vec/tree/main/src/modulesRandFunc>

Table 1: Hyperparameter settings of our models. In total, we trained four final models, which were configured systematically based on a pre-study. The number of randomly generated training instances is given per epoch, so, every model was trained on a total of 250 000 000 randomly generated single- and multi-objective functions.

Parameter Degree of Dim.	Medium Model		Large Model	
	25 <i>d</i>	50 <i>d</i>	25 <i>d</i>	50 <i>d</i>
ν	6	6	12	12
k	8	8	16	16
Num. MHA	6	6	6	6
Num. Heads	4	4	8	8
d_{model}	192	192	384	384
Epochs	250	250	250	250
Train Instances	1 000 000	1 000 000	1 000 000	1 000 000
Batch Size	1 264	632	1 264	632
Acc. Grad	1	2	1	2
τ	0.05	0.05	0.05	0.05
Stride	1	1	2	2
EMA Momentum	0.01	0.01	0.01	0.01
BN Momentum	0.1	0.1	0.1	0.1
Device	3× NVidia Q. RTX 6 000		3× NVidia RTX A 6 000	
GPU Memory	3× 24GB		3× 48GB	
Precision	Float16 (mixed)		BFloat16 (mixed)	
Params. BB.	2 263 296		9 189 888	
Params. Total	2 355 456		9 558 528	

to twelve total dimensions. The large model has twice the width of the medium model and produces twice the number of features (48 vs. 24). Due to the squared growth of transformer complexity with the number of tokens, we introduced a stride factor. A stride of two implies that every second token is omitted *after* the k NN-embedding, as depicted in Figure 4. Consequently, the large model’s complexity is reduced to a fourth, compared to a model with a stride of one, maintaining a larger batch size during the training of the large model. We then trained each model-variant with sample sizes of 25*d* and 50*d*, resulting in four final models, where d stands for the dimensions of $\mathcal{X}$. Hence, the sample-size scales linearly with the number of dimensions of the decision space. For the 50*d* models, we reduced the batch size by half but compensated with gradient accumulation over two iterations. However, InfoNCE does not benefit much from gradient accumulation since its efficiency relies on a substantial number of samples. Still, we opted for gradient accumulation to maintain a consistent number of update steps and training instances, as seen with the 25*d* models.

For training, we employed PyTorch-Lightning⁴ in tandem with PyTorch⁵. Post-training, we removed the two heads necessary during the training phase, resulting in a versatile backbone model suitable for various downstream tasks. Subsequently, we incorporated classical machine learners atop this backbone. No further fine-tuning of the backbone model was undertaken or deemed necessary. Instead, the backbone model can be directly applied as the considered studies have similar boxed constraints for $\mathcal{X}$ as the training instances. Yet, if e.g. the box constraints are noticeably dissimilar to the training set ($[-5, 5]$) fine-tuning may be advisable. But this requires further testing in future work. To emulate a user proficient in general machine learning but

⁴<https://lightning.ai/>

⁵<https://pytorch.org/>

Table 2: Assignment of the 24 single-objective noiseless BBOB functions into three frequently used *High-Level Properties* (HLP), as outlined and proposed in Mersmann et al. (2010) and Kerschke et al. (2015).

BBOB Function	Multim.	Global Str.	Funnel
1: Sphere	none	none	yes
2: Ellipsoidal separable	none	none	yes
3: Rastrigin separable	high	strong	yes
4: Büche-Rastrigin	high	strong	yes
5: Linear Slope	none	none	yes
6: Attractive Sector	none	none	yes
7: Step Ellipsoidal	none	none	yes
8: Rosenbrock	low	none	yes
9: Rosenbrock rotated	low	none	yes
10: Ellipsoidal high conditioned	none	none	yes
11: Discus	none	none	yes
12: Bent Cigar	none	none	yes
13: Sharp Ridge	none	none	yes
14: Different Powers	none	none	yes
15: Rastrigin multimodal	high	strong	yes
16: Weierstrass	high	med.	none
17: Schaffer F7	high	med.	yes
18: Schaffer F7 moderately ill-cond.	high	med.	yes
19: Griewank-Rosenbrock	high	strong	yes
20: Schwefel	med.	deceptive	yes
21: Gallagher 101 Peaks	med.	none	none
22: Gallagher 21 Peaks	low	none	none
23: Katsuura	high	none	none
24: Lunacek bi-Rastrigin	high	weak	yes

not deeply versed in deep learning specifics, we relied exclusively on `Scikit-Learn` and refrained from fine-tuning the backbone model. Further, we did not apply feature selection to demonstrate that the instance features generated by the Deep-ELA approach contain features of high relevance and little redundancy. Last, it should be noted that the backbone model did not receive any instances from the BBOB suite or similar frameworks and was solely trained on randomly generated instances. Further, the model receives each optimization instance exactly once. Hence, we argue that any overfitting to existing optimization problems is hardly possible. More importantly, as the models were not trained to predict certain labels as it is usually done in supervised learning, they cannot just remember labels for a specific task.

5.2 High-Level Property Prediction

The term *High-Level Properties* (HLP) refers to a collection of structural attributes describing the fitness landscape of the examined single-objective optimization. These properties enable the characterization of the problem – e.g., into low, medium, and high multimodality – facilitating the identification of (dis-)similar problems. This holds especially true for high-dimensional problems, which cannot be visually represented or analyzed. Moreover, these properties are pivotal for algorithm selection, algorithm configuration, and for crafting algorithms tailored to specific objectives. Of the eight properties detailed in Mersmann et al. (2011) and Kerschke et al. (2015), the following three exert the most pronounced influence on optimization problem complexity:

1. **Multimodality:** The *degree of multimodality* aggregates the number of local optima

Table 3: F1-Score with macro aggregation (F1-Scores a computed individually per class and then mean-aggregated over all classes) of predicting *High-Level Properties* of all 24 BBOB functions and instances 126 to 150. The results marked with a * are directly taken from Seiler et al. (2022) while the ones marked with ** are taken from van Stein et al. (2023). The last eight columns list the performances obtained by our proposed Deep-ELA models. van Stein et al. (2023) did not provide results for $d = 3$ and aggregated over all dimensions (all). Our medium models cannot handle data with more than six total dimensions. Therefore, there are no results for $d = 10$ for the medium models.

High-Level	Dim.	ELA* (50d)	Transformer* p100 p500		AE-32** (50d)	VAE-32** (50d)	Large (25d) RF SVM		Large (50d) RF SVM		Medium (25d) RF SVM		Medium (50d) RF SVM	
Multi-modality	2	0.997	0.991	0.997	0.849	0.856	0.854	0.845	0.895	0.904	0.896	0.905	0.938	0.939
	3	0.997	0.988	0.994	-/-	-/-	0.869	0.877	0.946	0.953	0.958	0.950	0.974	0.973
	5	0.999	0.991	0.999	0.903	0.889	0.922	0.942	0.945	0.953	0.959	0.966	0.961	0.966
	10	1.000	0.974	0.991	0.813	0.838	0.920	0.915	0.941	0.949	-/-	-/-	-/-	-/-
	all	0.998	0.986	0.995	-/-	-/-	0.893	0.896	0.932	0.940	0.938	0.940	0.958	0.959
Global-Structure	2	0.997	0.991	0.998	0.904	0.889	0.857	0.849	0.921	0.941	0.935	0.946	0.953	0.956
	3	0.996	0.986	0.994	-/-	-/-	0.890	0.909	0.954	0.968	0.960	0.961	0.976	0.977
	5	0.998	0.978	0.995	0.828	0.793	0.926	0.936	0.948	0.940	0.954	0.957	0.957	0.957
	10	0.999	0.963	0.984	0.737	0.745	0.898	0.887	0.933	0.953	-/-	-/-	-/-	-/-
	all	0.998	0.980	0.993	-/-	-/-	0.894	0.895	0.939	0.950	0.950	0.955	0.962	0.963
Funnel-Structure	2	0.998	0.999	1.000	0.974	0.978	0.986	0.980	0.994	0.994	0.988	0.991	0.994	0.991
	3	1.000	1.000	1.000	-/-	-/-	1.000	0.994	1.000	1.000	1.000	0.997	1.000	1.000
	5	1.000	1.000	1.000	1.000	0.991	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
	10	1.000	0.999	1.000	0.993	0.993	1.000	1.000	1.000	1.000	-/-	-/-	-/-	-/-
	all	1.000	1.000	1.000	-/-	-/-	0.996	0.994	0.999	0.999	0.996	0.996	0.998	0.997

into ‘low’, ‘medium’, and ‘high’ multimodality, as well as ‘none’ in case of uni-modal problems.

2. **Global Structure:** Chronicles the *structural* link between local and global optima.
3. **Funnel:** Signifies the presence of a *funnel*-like layout of the local and global optima.

The assignment of the 24 BBOB functions into the three HLPs is given in Table 2 for convenience. For predicting the HLPs, we adhered to the setup prescribed in Seiler et al. (2022). This entailed evaluating the 24 BBOB functions, adopting instances $\{1, \dots, 100\}$ for training and $\{125, \dots, 150\}$ for testing, with $d \in \{2, 3, 5, 10\}$. We juxtaposed our findings against those from Seiler et al. (2022). Additionally, we consider the results reported by van Stein et al. (2023) in the HLP study of their Doe2Vec method. For prediction purposes, we utilized *Random Forests* (RF) and *Support Vector Machines* (SVM) to forecast the three HLPs based on features generated by the pre-trained backbone model. The obtained (and collected) results are listed in Table 3.

As summarized in Table 3, the performances of our models lag behind those of Seiler et al. (2022) but are superior to the ones listed in van Stein et al. (2023). This was somewhat anticipated, given that Seiler et al. (2022) explicitly trained on BBOB instances for HLP prediction. In contrast, both van Stein et al. (2023) and our approach were trained by self-supervised learning on autonomously generated instances. Notably, our medium-scale models often outperformed their larger counterparts. Two potential reasons underpin this trend. Firstly, the large models, having been utilized with a stride of two, bypass half of the accessible data, potentially decreasing their efficacy. The second rationale suggests that while the larger models underwent an identical number of training iterations, they were exposed to a broader range of dimensions, possibly hindering their ability to generalize equally well to specific dimensions. Lastly, our models with 50d consistently outperformed the 25d models, echoing the findings of Seiler et al. (2022), who observed superior performance from transformers with a

sample size of 500 versus those with a size of 100. In terms of the considered machine learning models, our experiments did not discern any substantial performance discrepancy between the RF and SVM results; both exhibited roughly comparable efficiency.

5.3 Single-Objective Automated Algorithm Selection

In our subsequent case study, we both replicated and expanded upon the experiments presented by Prager et al. (2022). Integral to this study was the data originally compiled and introduced by Kerschke et al. (2015), encompassing benchmark results of twelve competitive, complementary algorithms – a comprehensive list of these algorithms can be found in Kerschke et al. (2015) and Prager et al. (2022). The benchmark performances were obtained from COCO, a repository archiving the outcomes of optimizer runs from a myriad of competitions hosted on the BBOB suites. In the methodology laid out by Kerschke et al. (2015), a threshold of 0.01 was deemed acceptable, i.e., an algorithm that procures a solution within 0.01 of the actual optimum (in objective space) is categorized as successful. An algorithm’s efficacy is gauged through the *relative Expected Running Time (relERT)* metric (Auger and Hansen, 2005). The relERT provides a relative variant of the ERT itself, by juxtaposing the ERT of a given algorithm against that of the *Virtual Best Solver* (VBS). Hence, a relERT of r indicates that the examined algorithm needs r times as many function evaluations compared to the best algorithm from the considered portfolio. Note that when predicting the top-performing algorithm, the size of the initial sample must be factored in as a cost, which is first added to the algorithm’s ERT prior to deriving the relERT. The sample is essentially needed for the model to scrutinize a given problem instance (in feature-free algorithm selection) and predominantly for feature computation (in feature-based algorithm selection).

Our experimental setup closely mirrored that of Prager et al. (2022), employing the 24 noiseless BBOB functions, instances $\{1, 2, 3, 4, 5\}$ with ten repetitions each, and $d \in \{2, 3, 5, 10\}$. In line with Prager et al. (2022), we executed five-fold cross-validation across the five instances for each function. Further, Prager et al. (2022) employed a cost-sensitive loss function, but it was exclusively compatible with machine learners amenable to gradient descent training. This discrepancy is evident when comparing the RF and MLP models, both trained on ELA features. While the RF model significantly underperformed in relation to the *Single Best Solver* (SBS), the MLP exhibited state-of-the-art results. Our research, however, sidestepped the loss function delineated by Prager et al. (2022), as we restricted ourselves to the `sklearn` framework, avoiding other platforms. In contrast, Prager et al. (2022) utilized `PyTorch` complemented by a bespoke loss function and training protocol. `sklearn`, however, does not provide the functionality to implement a custom loss function or to make use of an instance-based, cost-sensitive training routine.

Diverging from the insights of our initial case study, this research unveiled a noticeable enhancement in our proposed Deep-ELA methodology compared to the results of Prager et al. (2022). All our models consistently outperformed the feature-free strategies delineated in Prager et al. (2022). In certain scenarios, our Large-25d and Medium-50d models even surpassed the MLP-ELA model. A consistent observation from Prager et al. (2022) was the superior performance of the Medium-25d over the Medium-50d model. We postulate that the higher costs associated with acquiring supplementary information outweigh the benefits derived from this extra data. However, this paradigm does not apply to the large models, with the Large-50d model being superior to the Large-25d model. This trend suggests that larger models may be necessary to extract additional value from the added data of the larger sample size. Nevertheless, the dis-

Table 4: Comparison of the relative ERT (relERT) values for the algorithm selection study on instances 1 to 5 of the 24 single-objective BBOB functions. The first five columns (marked with *) are directly taken from Prager et al. (2022), while the last eight show the performance of our proposed Deep-ELA approach. Our medium models cannot handle data with more than six dimensions. Therefore, there are no results for $d = 10$ for the medium models.

D	F. Group	SBS*	ELA (50d)*		Transformer*		Large (25d)		Large (50d)		Medium (25d)		Medium (50d)	
			RF	MLP	p100	p500	kNN	RF	kNN	RF	kNN	RF	kNN	RF
2	1	3.71	10.41	10.59	23.95	61.15	9.24	10.30	14.26	13.87	17.57	7.57	16.38	14.77
	2	5.80	8.51	3.72	3.54	11.04	2.65	2.87	3.49	4.25	2.69	3.37	5.70	4.08
	3	6.29	1473.16	4.72	4.02	16.11	3.52	3.12	5.32	4.80	3.91	4.40	5.33	4.65
	4	25.34	3.89	9.25	8.90	12.21	6.45	9.76	5.64	6.76	5.73	5.99	3.48	4.29
	5	44.95	148.39	3.32	4.33	6.18	3.69	5.06	3.81	7.77	3.79	8.08	3.45	14.58
	all	17.69	342.22	6.43	9.17	21.77	5.21	6.36	6.63	7.62	6.91	5.99	6.92	8.66
3	1	356.10	1480.68	11.87	13.32	39.24	19.94	66.76	15.29	15.54	11.72	53.23	15.32	16.13
	2	4.46	8.33	3.50	2.85	6.65	2.73	3.02	3.74	3.75	2.67	2.87	3.42	3.62
	3	4.98	7.07	3.82	2.72	9.58	2.69	3.48	3.72	4.81	2.59	3.53	3.85	4.02
	4	2.63	441.96	5.06	11.49	11.87	5.15	4.63	5.20	5.00	5.36	4.83	5.12	4.20
	5	66.81	1.22	2.54	2.46	3.04	30.75	12.02	4.21	25.80	2.70	4.99	7.24	6.85
	all	90.43	403.67	5.44	6.72	14.39	12.65	18.61	6.54	11.28	5.10	14.35	7.14	7.10
5	1	11.99	14.14	11.97	16.27	33.39	17.70	17.31	22.88	22.81	17.50	17.85	24.18	24.01
	2	3.90	369.26	2.62	4.25	5.66	2.40	4.27	3.03	3.89	2.48	2.45	3.59	3.39
	3	4.21	150.44	3.97	5.21	9.23	3.70	4.87	4.63	6.29	3.76	3.69	4.33	4.58
	4	4.29	1470.28	6.81	4.33	4.47	1.87	4.07	1.90	4.24	3.95	3.51	2.25	2.44
	5	7.67	1.13	1.83	7.72	7.93	1.08	4.73	1.15	3.71	1.72	1.43	1.57	1.96
	all	6.52	402.38	5.56	7.69	12.41	5.47	7.17	6.87	8.37	6.02	5.93	7.33	7.44
10	1	2.74	14.64	15.27	5.46	16.34	9.54	9.53	16.45	16.28	-/-	-/-	-/-	-/-
	2	2.16	1.62	1.76	2.27	2.71	2.40	2.43	2.64	2.71	-/-	-/-	-/-	-/-
	3	2.76	2.87	4.35	3.10	4.48	3.54	3.62	4.52	4.15	-/-	-/-	-/-	-/-
	4	2.02	442.01	1.96	2.03	2.09	2.06	2.05	1.91	2.09	-/-	-/-	-/-	-/-
	5	23.64	148.01	3.25	23.66	23.74	1.73	11.33	1.80	12.09	-/-	-/-	-/-	-/-
	all	6.85	126.84	5.46	7.51	10.17	3.91	5.93	5.58	7.66	-/-	-/-	-/-	-/-
all	1	93.63	379.97	12.43	14.75	37.53	14.10	25.97	17.22	17.13	15.60	26.22	18.62	18.30
	2	4.08	96.93	2.90	3.23	6.52	2.54	3.15	3.23	3.65	2.62	2.90	4.24	3.69
	3	4.56	408.38	4.21	3.76	9.85	3.36	3.77	4.55	5.01	3.42	3.88	4.50	4.42
	4	8.57	589.54	5.77	6.69	7.66	3.88	5.13	3.66	4.52	5.01	4.78	3.62	3.64
	5	35.77	74.69	2.74	9.54	10.22	9.31	8.29	2.74	12.34	2.73	4.83	4.09	7.80
	all	30.37	318.78	5.72	7.78	14.68	6.81	9.52	6.41	8.73	6.01	8.76	7.13	7.73

parity between the results is nuanced. Our best-performing Deep-ELA model was the Medium-25d model, lending credence to the hypothesis that employing a stride of two may hamper performance. Notably, the k NN classifier substantially outperformed the RF classifier. In this context, we opted for a k value of 69, the sole hyperparameter we carefully fine-tuned manually.

Our investigations into single-objective AAS unveiled a pivotal revelation: Deep-ELA operates as envisaged, particularly excelling with limited datasets compared to feature-free AAS, where the propensity for overfitting makes training extensive deep-learning models a challenge.

5.4 Multi-Objective Automated Algorithm Selection

Our experimental design is akin to that of Rook et al. (2022), albeit without the *algorithm configuration* segment. Instead, we opted for the seven algorithms (as detailed in Rook et al., 2022) in their default settings and employed an AAS approach to select the optimal algorithm. Three of these seven algorithms – NSGA-II (Deb et al.,

Table 5: The mean relative HV (relHV) of our proposed Deep-ELA approach on the multi-objective AAS study. relHV values are calculated per test instance and mean-aggregated first over instances of the same function and then over functions of the same group. A value of one (zero) indicates performances comparable to the VBS (SBS). Negative values indicate a performance worse than the SBS.

Group	Large (25d)		Large (50d)		Medium (25d)		Medium (50d)	
	kNN	RF	kNN	RF	kNN	RF	kNN	RF
BiBBOB	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
DTL	0.914	0.457	1.000	0.651	1.000	0.651	1.000	0.822
MMF	0.892	0.463	0.931	0.609	0.929	0.764	1.000	0.933
ZDT	-0.079	-0.087	0.041	0.209	0.688	0.185	0.616	0.745
all	0.682	0.458	0.743	0.617	0.904	0.650	0.904	0.875

2002), SMS-EMOA (Beume et al., 2007), and MOEA/D (Zhang and Li, 2007) – represent classical *Evolutionary Multiobjective Optimization Algorithms* (EMOAs), while the other four are Omni-Optimizer (Deb and Tiwari, 2005), MOLE (Schäpermeier, 2022), MOGSA (Grimme et al., 2019), and HIGA-MO (Wang et al., 2017). Performance was gauged using the *Hypervolume* (HV) of the approximated Pareto front found within a budget of 20 000 function evaluations. For most test instances, the reference point for the HV was pre-specified. For problems without predetermined reference points (for HV computation), we derived the necessary points from the least favorable solution of all algorithms for that specific instance. To maintain HV value consistency across distinct instances, we normalized the HV values, basing them on the highest HV identified from additional runs of all algorithms with a more substantial budget of 100 000 function evaluations. Subsequently, the resulting normalized HV values were contrasted against the SBS-VBS gap – and we coined this metric relative HV (relHV). In this context, a value close to zero resembles SBS performance, while a value of one relates to VBS performance. Negative values, in turn, denote performances that are inferior to the SBS. Formally, the metric is defined as

$$\text{relHV}(I, A) = \frac{\text{HV}(I, A) - \text{HV}_{\text{SBS}} + 10^{-8}}{\text{HV}_{\text{VBS}} - \text{HV}_{\text{SBS}} + 10^{-8}} \quad (12)$$

where 10^{-8} is a tiny value that prevents division by zero. This could happen if the SBS and VBS are identical. By also including the term 10^{-8} in the numerator ensures that the relHV is one, in case the selector predicts the VBS.

The dataset comprised ZDT, DTLZ, and MMF test instances, excluding ZDT5 and MMF13. Additionally, instances f_{46} , f_{47} , and f_{50} from the Bi-BBOB were integrated. This resulted in a total of 33 instances – with a two-dimensional decision ($d = 2$) and objective ($m = 2$) space each. We executed 20 repetitions per instance, of which the first 15 were used for training and the remaining five (16 – 20) for testing.

The results of our exploration are listed in Table 5. Due to the lack of other AAS studies on multi-objective optimization, we cannot compare Deep-ELA to any other studies from the literature. A potential reason for this may be the somewhat limited applicability of ELA on multi-objective optimization problems. It is pertinent to note that the crux of this study revolves around maximizing the relHV. Given the absence of any costs to factor in for this case study – as we measure the performance based

on the found hypervolume instead of the number of function evaluations – achieving performances on the VBS level is viable. Our findings reveal that the medium models eclipse the larger ones in performance, and the $50d$ models have a performance edge over the $25d$ models. This observation aligns with our expectations, considering that the $50d$ models harness more information than their $25d$ counterparts. We posit that the larger models are potentially hampered by the stride of two. Specifically, the large $25d$ model registered a negative relVH value for the ZDT instance group, suggesting a performance that lags behind the SBS. The k NN classifiers showcased a discernibly superior performance in comparison to the RF classifiers. In this context, we opted for a smaller $k = 15$. Both, the Medium $25d$ and Medium $50d$ models rendered comparable outcomes when leveraging a k NN classifier. However, when an RF classifier was in play, the Medium $50d$ model emerged as the top performer. In conclusion, the performances achieved by our Deep-ELA approach in this study are often very good – and, in many cases, even perfect.

6 Discussion & Conclusion

A salient outcome of this research is the demonstrated efficacy of the Deep-ELA methodology. It can either be used out-of-the-box for analyzing single- and multi-objective continuous optimization problems, as demonstrated, or fine-tuned to various tasks on algorithm behavior and problem understanding. In comparison to the strategies delineated by Prager et al. (2022), Deep-ELA exhibited significant performance enhancements, particularly in its adept handling of limited datasets. Feature-free deep-learning models, like those utilized by Prager et al. (2022), often contend with overfitting challenges when trained on limited datasets. In contrast, Deep-ELA’s design, having been trained on millions of randomly generated optimization instances, offers a remedy to such constraints. As a result, the backbone model could be integrated seamlessly into existing and novel downstream tasks without necessitating additional training or fine-tuning on the dataset from Prager et al. (2022). Furthermore, the derived instance features could be employed without the need for feature selection or normalization, as these features are less correlated than the commonly used ELA features, and, in addition, are all located within $[-1, 1]$.

Yet, alongside these advantages, the investigations revealed certain intricacies warranting further examination. Notably, medium-sized models demonstrated superior performance compared to their larger counterparts. Concurrently, a trend emerged indicating that models with a sample size of $50d$ performed better than those with $25d$. While the latter trend is intuitive – a sample size of $50d$ inherently offers more information for the model to process – the former raises questions regarding the current iteration of the Deep-ELA design. The use of a stride of two, particularly in larger models, was pinpointed as a possible bottleneck inhibiting peak performance. This observation offers a compelling avenue for further research. Even though the chosen stride of size two was indispensable for accommodating large training batches, future studies could explore more refined pooling techniques to diminish sequence sizes without significant data loss. Another potential explanation for the diminished performance of larger models may lie in their training on a more varied set of decision space dimensions. As a future direction, researchers may delve into whether extended training durations would amplify results or if other underlying factors are at play.

Despite the contained scope of the results, the Deep-ELA methodology made a noteworthy foray into the multi-objective AAS domain. The capability of Deep-ELA to seamlessly transition to multi-objective problem instances stands out as a significant

achievement. This is particularly notable considering that traditional ELA features are innately tailored for single-objective problems. Moving forward, there is ample opportunity to devise more intricate studies where the Deep-ELA approach can be further tested and refined. At last, configuring the hyperparameters of the Deep-ELA framework in an automated manner, e.g., using efficient algorithm configurators such as irace (López-Ibáñez et al., 2016) or SMAC (Lindauer et al., 2022), also provides a very promising direction for future research.

References

- Alissa, M., Sim, K., and Hart, E. (2019). Algorithm Selection Using Deep Learning without Feature Extraction. In *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM.
- Auger, A. and Hansen, N. (2005). Performance evaluation of an advanced local search evolutionary algorithm. In *2005 IEEE congress on evolutionary computation*, volume 2, pages 1777–1784. IEEE.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Beume, N., Naujoks, B., and Emmerich, M. (2007). Sms-emoa: Multiobjective selection based on dominated hypervolume. *European Journal of Operational Research*, 181(3):1653–1669.
- Breiman, L. (2001). Random Forests. *Machine learning*, 45:5–32.
- Brockhoff, D., Auger, A., Hansen, N., and Tušar, T. (2022). Using Well-Understood Single-Objective Functions in Multiobjective Black-Box Optimization Test Suites. *Evolutionary Computation*, 30(2):165–193.
- Cenikj, G., Petelin, G., and Eftimov, T. (2023). TransOpt: Transformer-based Representation Learning for Optimization Problem Classification.
- Chen, X., Xie, S., and He, K. (2021). An empirical study of training self-supervised vision transformers. *arXiv preprint arXiv:2104.02057*.
- Cortes, C. and Vapnik, V. (1995). Support-Vector Networks. *Machine learning*, 20:273–297.
- Cover, T. and Hart, P. (1967). Nearest Neighbor Pattern Classification. *IEEE transactions on information theory*, 13(1):21–27.
- Dauphin, Y. N., Fan, A., Auli, M., and Grangier, D. (2017). Language modeling with gated convolutional networks. In *International conference on machine learning*, pages 933–941. PMLR.
- Deb, K., Pratap, A., Agarwal, S., and Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE transactions on evolutionary computation*, 6(2):182–197.
- Deb, K., Thiele, L., Laumanns, M., and Zitzler, E. (2005). Scalable Test Problems for Evolutionary Multiobjective Optimization. In Abraham, A., Jain, L., and Goldberg, R., editors, *Evolutionary Multiobjective Optimization*, Advanced Information and Knowledge Processing (AI & KP), pages 105 – 145. Springer.
- Deb, K. and Tiwari, S. (2005). Omni-optimizer: A procedure for single and multi-objective optimization. In *International conference on evolutionary multi-criterion optimization*, pages 47–61. Springer.
- Eftimov, T., Popovski, G., Renau, Q., Korošec, P., and Doerr, C. (2020). Linear Matrix Factorization Embeddings for Single-objective Optimization Landscapes. In *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 775 – 782. IEEE.
- Geva, M., Schuster, R., Berant, J., and Levy, O. (2020). Transformer feed-forward layers are key-value memories. *arXiv preprint arXiv:2012.14913*.

- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P., Buchatskaya, E., Doersch, C., Avila Pires, B., Guo, Z., Gheshlaghi Azar, M., et al. (2020). Bootstrap your own latent-a new approach to self-supervised learning. *Advances in neural information processing systems*, 33:21271–21284.
- Grimme, C., Kerschke, P., and Trautmann, H. (2019). Multimodality in multi-objective optimization—more boon than bane? In *Proceedings of the 10th International Conference on Evolutionary Multi-Criterion Optimization*, pages 126–138. Springer.
- Guo, M.-H., Cai, J.-X., Liu, Z.-N., Mu, T.-J., Martin, R. R., and Hu, S.-M. (2021). PCT: Point Cloud Transformer. *Computational Visual Media*, 7(2):187–199.
- Hansen, N., Brockhoff, D., Mersmann, O., Tusar, T., Tusar, D., ElHara, O. A., Sampaio, P. R., Atamna, A., Varelas, K., Batu, U., Nguyen, D. M., Matzner, F., and Auger, A. (2019). Comparing Continuous Optimizers: numbbo/COCO on Github.
- Hansen, N., Finck, S., Ros, R., and Auger, A. (2009). Real-Parameter Black-Box Optimization Benchmarking 2009: Noiseless Functions Definitions. Research Report RR-6829, INRIA.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.
- Huang, C., Li, Y., and Yao, X. (2019). A survey of automatic parameter tuning methods for metaheuristics. *IEEE transactions on evolutionary computation*, 24(2):201–216.
- Hutter, F., Hoos, H. H., Leyton-Brown, K., and Stützle, T. (2009). Paramils: an automatic algorithm configuration framework. *Journal of artificial intelligence research*, 36:267–306.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr.
- Jones, T., Forrest, S., et al. (1995). Fitness distance correlation as a measure of problem difficulty for genetic algorithms. In *ICGA*, volume 95, pages 184–192.
- Kerschke, P., Hoos, H. H., Neumann, F., and Trautmann, H. (2019). Automated algorithm selection: Survey and perspectives. *Evolutionary computation*, 27(1):3–45.
- Kerschke, P. and Preuss, M. (2023). Exploratory landscape analysis. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO) Companion*, pages 990 – 1007.
- Kerschke, P., Preuss, M., Wessing, S., and Trautmann, H. (2015). Detecting funnel structures by means of exploratory landscape analysis. In *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation*, pages 265–272.
- Kerschke, P. and Trautmann, H. (2016). The R-Package FLACCO for Exploratory Landscape Analysis with Applications to Multi-Objective Optimization Problems. In *Proceedings of the IEEE Congress on Evolutionary Computation (CEC)*, pages 5262 – 5269. IEEE.
- Kerschke, P. and Trautmann, H. (2019). Comprehensive feature-based landscape analysis of continuous and constrained optimization problems using the r-package flacco. *Applications in Statistical Computing: From Music Data Analysis to Industrial Quality Improvement*, pages 93–123.
- LeCun, Y. and Bengio, Y. (1995). Convolutional Networks for Images, Speech, and Time Series. *The Handbook of Brain Theory and Neural Networks*, 3361(10):1995.
- Lindauer, M., Eggensperger, K., Feurer, M., Biedenkapp, A., Deng, D., Benjamins, C., Ruhkopf, T., Sass, R., and Hutter, F. (2022). Smac3: A versatile bayesian optimization package for hyperparameter optimization. *Journal of Machine Learning Research*, 23(54):1 – 9.
- López-Ibáñez, M., Dubois-Lacoste, J., Cáceres, L. P., Birattari, M., and Stützle, T. (2016). The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43 – 58.

- Lunacek, M. and Whitley, D. (2006). The dispersion metric and the cma evolution strategy. In *Proceedings of the 8th annual conference on Genetic and evolutionary computation*, pages 477–484.
- McKay, D., Beckman, R., and Conover, W. (1979). A comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics* 21.
- Mersmann, O., Bischl, B., Trautmann, H., Preuss, M., Weihs, C., and Rudolph, G. (2011). Exploratory landscape analysis. In *Proceedings of the 13th annual conference on Genetic and evolutionary computation*, pages 829–836.
- Mersmann, O., Preuss, M., and Trautmann, H. (2010). Benchmarking evolutionary algorithms: Towards exploratory landscape analysis. In *Parallel Problem Solving from Nature, PPSN XI: 11th International Conference, Kraków, Poland, September 11-15, 2010, Proceedings, Part I* 11, pages 73–82. Springer.
- Muñoz, M. A., Kirley, M., and Halgamuge, S. K. (2014). Exploratory landscape analysis of continuous space optimization problems using information content. *IEEE transactions on evolutionary computation*, 19(1):74–87.
- Nguyen, T. Q. and Salazar, J. (2019). Transformers without tears: Improving the normalization of self-attention. *arXiv preprint arXiv:1910.05895*.
- Oord, A. v. d., Li, Y., and Vinyals, O. (2018). Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*.
- Prager, R. P., Dietrich, K., Schneider, L., Schäpermeier, L., Bischl, B., Kerschke, P., Trautmann, H., and Mersmann, O. (2023). Neural networks as black-box benchmark functions optimized for exploratory landscape features. In *Proceedings of the 17th ACM/SIGEVO Conference on Foundations of Genetic Algorithms*, pages 129–139.
- Prager, R. P., Seiler, M. V., Trautmann, H., and Kerschke, P. (2021). Towards feature-free automated algorithm selection for single-objective continuous black-box optimization. In *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE.
- Prager, R. P., Seiler, M. V., Trautmann, H., and Kerschke, P. (2022). Automated algorithm selection in single-objective continuous optimization: A comparative study of deep learning and landscape analysis methods. In *Parallel Problem Solving from Nature—PPSN XVII: 17th International Conference, PPSN 2022*. Springer.
- Prager, R. P. and Trautmann, H. (2023). Nullifying the inherent bias of non-invariant exploratory landscape analysis features. In *International Conference on the Applications of Evolutionary Computation (Part of EvoStar)*, pages 411–425. Springer.
- Renau, Q., Doerr, C., Dreó, J., and Doerr, B. (2020). Exploratory Landscape Analysis is Strongly Sensitive to the Sampling Strategy. pages 139 – 153.
- Renau, Q., Dréo, J., Doerr, C., and Doerr, B. (2019). Expressiveness and Robustness of Landscape Features. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO) Companion*, pages 2048 – 2051. ACM.
- Rice, J. R. (1976). The Algorithm Selection Problem. *Advances in Computers*, 15.
- Rook, J., Trautmann, H., Bossek, J., and Grimme, C. (2022). On the potential of automated algorithm configuration on multi-modal multi-objective optimization problems. In *Proceedings of the Genetic and Evolutionary Computation Conference Companion*, pages 356–359.
- Schede, E., Brandt, J., Tornede, A., Wever, M., Bengs, V., Hüllermeier, E., and Tierney, K. (2022). A survey of methods for automated algorithm configuration. *Journal of Artificial Intelligence Research*, 75:425 – 487.
- Schäpermeier, L. (2022). An R Package Implementing the Multi-Objective Landscape Explorer (MOLE).

- Seiler, M., Škvorc, U., Doerr, C., and Trautmann, H. (2024). Synergies of Deep and Classical Exploratory Landscape Features for Automated Algorithm Selection. In *Learning and Intelligent Optimization - 18th International Conference, LION 18, Ischia Island, Italy, June 9-13, 2024, (accepted as full paper)*. Springer.
- Seiler, M. V., Pohl, J., Bossek, J., Kerschke, P., and Trautmann, H. (2020). Deep learning as a competitive feature-free approach for automated algorithm selection on the traveling salesperson problem. In *Parallel Problem Solving from Nature-PPSN XVI: 16th International Conference, PPSN 2020*. Springer.
- Seiler, M. V., Prager, R. P., Kerschke, P., and Trautmann, H. (2022). A collection of deep learning-based feature-free approaches for characterizing single-objective continuous fitness landscapes. In *Proceedings of the Genetic and Evolutionary Computation Conference*.
- Smith-Miles, K., Baatar, D., Wreford, B., and Lewis, R. (2014). Towards objective measures of algorithm performance across instance space. *Computers & Operations Research*, 45:12–24.
- Smith-Miles, K. and Muñoz, M. A. (2023). Instance space analysis for algorithm testing: Methodology and software tools. *ACM Computing Surveys*, 55(12):1–31.
- Sobol', I. M. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *Zhurnal Vychislitel'noi Matematiki i Matematicheskoi Fiziki*, 7(4):784–802.
- Tian, Y., Peng, S., Zhang, X., Rodemann, T., Tan, K. C., and Jin, Y. (2020). A recommender system for metaheuristic algorithms for continuous optimization based on deep recurrent neural networks. *IEEE transactions on artificial intelligence*, 1(1):5–18.
- van Stein, B., Long, F. X., Frenzel, M., Krause, P., Gitterle, M., and Bäck, T. (2023). Doe2vec: Deep-learning based features for exploratory landscape analysis. *arXiv preprint arXiv:2304.01219*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- Volz, V., Naujoks, B., Kerschke, P., and Tušar, T. (2023). Tools for landscape analysis of optimisation problems in procedural content generation for games. *Applied Soft Computing*, 136:110121.
- Wang, H., Deutz, A., Bäck, T., and Emmerich, M. (2017). Hypervolume indicator gradient ascent multi-objective optimization. In *Evolutionary Multi-Criterion Optimization: 9th International Conference, EMO 2017, Münster, Germany, March 19-22, 2017, Proceedings 9*, pages 654–669. Springer.
- Zhang, Q. and Li, H. (2007). Moea/d: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on evolutionary computation*, 11(6):712–731.
- Zitzler, E., Deb, K., and Thiele, L. (2000). Comparison of Multiobjective Evolutionary Algorithms: Empirical Results. *Evolutionary Computation (ECJ)*, (2):173 – 195.